

Module 4

GAN, Vision Transformers & Déploiement

Apprentissage adversarial — attention visuelle — mise en production

Du génératif adversarial à la production

Générer — transformer par l'attention — déployer — projet fil rouge

$\min_G \max_D$

GAN

DCGAN, Pix2Pix, SRGAN

QK^TV

ViT & Swin

Self-attention



Déploiement

TorchScript, ONNX



Projet

Pipeline complet

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

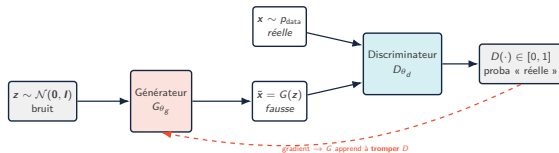
ENETCOM — Université de Sfax

Présentation 04 — Module 4

1. GAN : le jeu minimax & ses variantes
2. Vision Transformers : ViT, Swin & SegFormer
3. Déploiement : TorchScript, ONNX & quantification
4. Projet fil rouge & synthèse

GAN : le jeu minimax & ses variantes

GAN — principe : le jeu minimax



L'objectif minimax (Goodfellow et al., 2014) — la formule à retenir

Deux réseaux s'affrontent sur la **même** valeur V : D la maximise, G la minimise.

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]}_{\text{(A) sur les images réelles}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{\text{(B) sur les images générées}}$$

- (A) : D pousse $D(x) \rightarrow 1$ sur le réel ; (B) : D pousse $D(G(z)) \rightarrow 0$ sur le faux $\Rightarrow D$ **maximise** ($\mathbb{E}[\cdot]$: moyenne sur un batch) ;
- G n'apparaît que dans (B) : il pousse $D(G(z)) \rightarrow 1$ pour tromper $D \Rightarrow G$ **minimise** ;
- **équilibre de Nash** : $p_g = p_{\text{data}}$ et $D(\cdot) = \frac{1}{2}$ partout — le **faussaire** (G) produit des billets parfaits, l'**expert** (D) est réduit au hasard.

Calcul à la main — une image réelle, une image générée

D rend $D(x) = 0.9$ et $D(G(z)) = 0.3$: $V = \log 0.9 + \log(1 - 0.3) = -0.105 - 0.357 = \mathbf{-0.462}$. D parfait : $\log 1 + \log 1 = 0$; G qui trompe parfaitement : $D(G(z)) \rightarrow 1 \Rightarrow \log(1 - 1) \rightarrow -\infty$.

GAN — le discriminateur optimal D^*

À G fixé, le meilleur discriminateur possible s'écrit explicitement

$$D^*(x) = \frac{\overbrace{p_{\text{data}}(x)}^{\text{densité du réel en } x}}{\underbrace{p_{\text{data}}(x) + p_g(x)}_{\text{densité totale : réel + généré}}} = \text{la « part du réel » au point } x.$$

Calcul à la main

En un point x où $p_{\text{data}}(x) = 0.8$ et $p_g(x) = 0.2$: $D^*(x) = \frac{0.8}{0.8 + 0.2} = \mathbf{0.8}$ — 80 % de chances que l'image vienne du réel.

Si G copie parfaitement les données ($p_g = p_{\text{data}}$) : $D^* = \frac{p}{p + p} = \mathbf{0.5 \text{ partout}}$ — l'expert ne fait plus mieux que pile ou face.

Ce que G minimise vraiment : un écart entre distributions

En remplaçant D par D^* dans V , la loss de G devient (à une constante près) la **divergence de Jensen-Shannon** $\text{JSD}(p_{\text{data}} \parallel p_g)$: une mesure **symétrique** de l'écart entre les deux distributions, nulle uniquement si $p_g = p_{\text{data}}$, plafonnée à $\log 2$.

Lecture : entraîner D **mesure** l'écart réel/généré ; entraîner G le **réduit**.

⚠ Si les deux distributions ne se recouvrent pas (début d'entraînement typique), la JSD reste **bloquée à $\log 2$** : plus aucun gradient pour G — motivation de la loss non saturante et du WGAN.

GAN — lire les courbes d'entraînement

Un GAN ne *minimise* pas une courbe

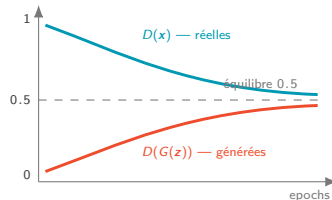
Aucune des deux losses ne doit tendre vers 0 : on cherche un **équilibre**, pas un minimum. Valeurs cibles à l'équilibre ($p_g = p_{\text{data}}$) :

$$D(x) \approx D(G(z)) \approx 0.5, \quad \mathcal{L}_D \approx 2 \log 2 \approx 1.386.$$

Calcul à la main — d'où vient 1.386 ?

À l'équilibre, D répond 0.5 partout :

$$\mathcal{L}_D = -\log \underbrace{0.5}_{D(x)} - \log \left(1 - \underbrace{0.5}_{D(G(z))} \right) = 0.693 + 0.693 = \mathbf{1.386}.$$



Le vrai juge : les images

Suivre une **grille générée sur un bruit fixe z_{fix}** d'epoch en epoch — les courbes ne mesurent pas la qualité visuelle.

Trois signatures d'échec

- $D(x) \rightarrow 1$ et $D(G(z)) \rightarrow 0$: **D a gagné** — gradients évanescents, G n'apprend plus (slide suivante) ;
- le scénario inverse : **G domine** un discriminateur trop faible — le jeu ne mesure plus rien ;
- des z différents donnent **la même image** : *mode collapse*, diversité effondrée — motivation du WGAN.

GAN — loss saturante vs non saturante : exemple chiffré

Astuce non saturante (Goodfellow et al., 2014)

En pratique G ne minimise pas $\log(1 - D(G(z)))$ mais **maximise** $\log D(G(z))$ — même point fixe, gradients opposés en début d'entraînement. Avec $D = \sigma(s)$ (s : logit), les gradients par rapport à s valent :

$$\frac{\partial}{\partial s} \log(1 - \sigma(s)) = -\sigma(s), \quad \frac{\partial}{\partial s} \log \sigma(s) = 1 - \sigma(s).$$

Exemple numérique — un pas d'update

Logits du discriminateur : $s_{\text{réel}} = 2.2$, $s_{\text{faux}} = -0.85$:

$$D(x) = \sigma(2.2) = 0.900, \quad D(G(z)) = \sigma(-0.85) = 0.299$$

$$\begin{aligned} \mathcal{L}_D &= -[\log 0.900 + \log(1 - 0.299)] \\ &= 0.105 + 0.355 = \mathbf{0.460} \text{ nats.} \end{aligned}$$

Gradient reçu par G (via s_{faux}) :

$$\text{saturante : } -\sigma(-0.85) = -0.299$$

$$\text{non sat. : } 1 - \sigma(-0.85) = \mathbf{+0.701}.$$

Début d'entraînement : $D(G(z)) = 0.02$

G produit du bruit, D le repère facilement :

$$\text{saturante : } -0.02 \quad (\text{évanescent !})$$

$$\text{non sat. : } 1 - 0.02 = \mathbf{0.98} \quad (\text{fort signal})$$

C'est **quand G est mauvais** que la version non saturante envoie le plus de gradient — exactement ce qu'il faut pour démarrer.

En PyTorch

La loss non saturante de G = `BCEWithLogitsLoss` avec les **labels « réels »** sur les images fausses (slide suivante).

GAN — boucle d'entraînement PyTorch

```
bce = nn.BCEWithLogitsLoss()
for x_real, _ in loader:
    # --- 1) Update du DISCRIMINATEUR ---
    z = torch.randn(b, 100, device=dev)
    x_fake = G(z)
    d_real = D(x_real)
    d_fake = D(x_fake.detach()) # PAS de grad vers G
    loss_D = bce(d_real, torch.ones_like(d_real)) \
        + bce(d_fake, torch.zeros_like(d_fake))
    opt_D.zero_grad(); loss_D.backward(); opt_D.step()

    # --- 2) Update du GENERATEUR ---
    d_fake = D(x_fake) # graphe complet z->G->D
    loss_G = bce(d_fake, torch.ones_like(d_fake))
    opt_G.zero_grad(); loss_G.backward(); opt_G.step()
```

Les 3 points critiques

1. `x_fake.detach()` dans l'update de D : sinon le `backward` de $\mathcal{L}_D$ remonterait dans G ;
2. **deux `backward()` distincts**, deux optimiseurs (`opt_D`, `opt_G`) ;
3. labels « réels » (**ones**) sur les fausses images pour $\mathcal{L}_G$: c'est la loss **non saturante**.

Hyperparamètres robustes (recette DCGAN)

`Adam(lr=2e-4, betas=(0.5, 0.999))` pour G et D — β_1 abaissé : moins d'inertie dans un jeu non stationnaire ; init $\mathcal{N}(0, 0.02)$ des poids conv et BatchNorm ; alternance stricte 1 update D / 1 update G .

DCGAN — règles architecturales convolutives

Pipeline du générateur (Radford et al., 2016) — $z \in \mathbb{R}^{100}$ vers une image $64 \times 64 \times 3$

$z(100) \xrightarrow{\text{ConvTranspose2d}} 4 \times 4 \times 512 \xrightarrow{k4, s2} 8 \times 8 \times 256 \xrightarrow{k4, s2} 16 \times 16 \times 128 \xrightarrow{k4, s2} 32 \times 32 \times 64 \xrightarrow{k4, s2, \tanh} 64 \times 64 \times 3$

Les règles DCGAN

Élément	Générateur	Discriminateur
Échantillonnage	<code>ConvTranspose2d</code> $k=4, s=2$	<code>Conv2d</code> $s=2$ (pas de pooling)
Normalisation	<code>BatchNorm2d</code> (sauf sortie)	<code>BatchNorm2d</code> (sauf entrée)
Activation	<code>ReLU</code>	<code>LeakyReLU(0.2)</code>
Sortie	$\tanh([-1, 1])$	logit (1 scalaire)
Couches denses	aucune	aucune

Pourquoi ces choix ?

- $\tanh$ en sortie de $G \Leftrightarrow$ données normalisées dans $[-1, 1]$ (comme la diffusion, Module 3) ;
- `LeakyReLU` dans D : un gradient subsiste même pour les activations négatives — G en dépend ;
- $k=4, s=2$: k multiple de $s \Rightarrow$ pas d'artefacts en damier (cf. Module 3) ;
- `BatchNorm` stabilise le jeu adversarial (évite l'effondrement des statistiques).

DCGAN — l'espace latent est une carte continue

Interpolation latente : le test de généralisation

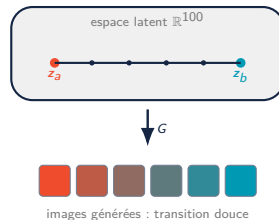
Entre deux bruits z_a et z_b , on parcourt le segment :

$$z_\alpha = \underbrace{(1 - \alpha) z_a + \alpha z_b}_{\alpha \in [0,1] : \text{ curseur de } a \text{ vers } b} \quad \text{puis on regarde } G(z_\alpha).$$

Si chaque image intermédiaire est **plausible** et la transition **douce**, G a appris une **variété continue** d'images — il ne mémorise pas le jeu d'entraînement.

Calcul à la main

$\alpha = 0.3 : z_{0.3} = 0.7 z_a + 0.3 z_b$ — 30 % du chemin vers z_b . Avec 10 valeurs de α (0, 0.1, ..., 1) : un **morphing** en 10 images.



Arithmétique latente

Les attributs sont des **directions** : $G(z + \bar{z}_{\text{sombre}} - \bar{z}_{\text{clair}})$ assombrit l'image générée ($\bar{z}$: moyenne des bruits d'images sombres / claires).

*Détail utile : en dimension $d = 100$, $\|z\| \approx \sqrt{d} = 10$ — le milieu du segment a une norme anormalement faible ; l'interpolation **sphérique** (slerp) respecte la norme.*

WGAN-GP — distance de Wasserstein & gradient penalty

Changer de distance (Arjovsky et al., 2017)

La JSD dit seulement **si** les distributions diffèrent ; la **distance de Wasserstein** W dit **à quelle distance** elles sont : le coût minimal pour transporter la masse de p_g vers p_{data} . Forme utilisée en pratique (dualité de Kantorovich-Rubinstein) :

$$W(p_{\text{data}}, p_g) = \max_{f \text{ 1-Lipschitz}} \underbrace{\mathbb{E}_{p_{\text{data}}} [f(\mathbf{x})]}_{\text{score moyen sur réel}} - \underbrace{\mathbb{E}_{p_g} [f(\tilde{\mathbf{x}})]}_{\text{score moyen sur généré}}$$

Terme à terme : f = le **critique**, un réseau qui *note* les images (score réel, **sans sigmoïde**) ; **1-Lipschitz** = pente bornée, $|f(\mathbf{a}) - f(\mathbf{b})| \leq \|\mathbf{a} - \mathbf{b}\|$ (sinon le max explose) ; W = le plus grand écart de score moyen que le critique peut creuser entre réel et généré.

Calcul à la main — en 1D

Tout le réel en $x = 0$, tout le généré en $x = d$. Transporter la masse coûte $d \Rightarrow W = d$. Vérification par le critique $f(x) = -x$ (pente -1 , admissible) :

$$\mathbb{E}[f(\text{réel})] - \mathbb{E}[f(\text{gén.})] = 0 - (-d) = d. \checkmark$$

La JSD vaut $\log 2$ que $d = 0.1$ ou $d = 100$ (aucun signal) ; $W = d$ décroît quand G se rapproche : **gradient utile en permanence**.

Imposer 1-Lipschitz : la gradient penalty

Gulrajani et al., 2017 — tirer la pente du critique vers 1 sur des points $\hat{\mathbf{x}}$ interpolés entre réel et généré :

$$\mathcal{L}_{\text{GP}} = \lambda \mathbb{E}_{\hat{\mathbf{x}}} \left[\left(\underbrace{\|\nabla_{\hat{\mathbf{x}}} f(\hat{\mathbf{x}})\|_2}_{\text{pente locale}} - \underbrace{1}_{\text{cible}} \right)^2 \right], \quad \lambda=10.$$

En pratique

- loss du critique $\approx -\hat{W}$: une courbe qui **suit enfin la qualité visuelle** ; **mode collapse** fortement réduit ;
- coût : $n_{\text{critic}} = 5$ updates du critique par update de G ; **InstanceNorm2d** plutôt que BatchNorm (pénalité par échantillon).

cGAN — génération conditionnelle

Conditionner le jeu par une variable y (Mirza & Osindero, 2014)

Classe, attribut, ou même une image entière — y est fourni **aux deux joueurs** :

$$\min_G \max_D \mathbb{E}_{x,y} [\log D(x | y)] + \mathbb{E}_{z,y} [\log (1 - D(G(z | y)))] .$$

Même jeu minimax qu'avant, une seule différence : chaque joueur voit y en plus — $G(z | y)$ génère *pour* cette condition, $D(x | y)$ ne juge plus « est-ce réel ? » mais « **est-ce réel et cohérent avec y ?** » (une image réaliste avec le mauvais label doit être rejetée).

Implémentation PyTorch

- `label` → `nn.Embedding(n_classes, d)` ;
- dans G : embedding **concaténé à z** ;
- dans D : embedding projeté en carte spatiale et **concaténé aux canaux de x** ;
- même boucle d'entraînement, mêmes pertes.

Cas d'usage

- génération **par classe** : équilibrer un dataset médical (lésions rares) ;
- synthèse contrôlée pour tests : « générer un défaut industriel de type k » ;
- quand y est une **image** : on obtient la traduction d'image → **Pix2Pix** (slide suivante).

Déjà vu au Module 3

Même principe que le **CVAE** et la diffusion conditionnelle — le conditionnement par embedding est un motif transversal du génératif.

Pix2Pix — traduction d'image avec PatchGAN

Objectif hybride (Isola et al., 2017) : adversarial + fidélité L_1

Paires alignées (x, y) , générateur = **U-Net** (Module 3), loss à deux termes :

$$\mathcal{L}_{\text{Pix2Pix}} = \underbrace{\mathcal{L}_{\text{cGAN}}}_{\text{jeu adversarial conditionné par } x \text{ netteté, textures (hautes fréquences)}} + \lambda \underbrace{\mathbb{E}[\|y - G(x)\|_1]}_{\substack{\text{écart pixel moyen à la cible} \\ \text{fidélité globale (basses fréquences)}}, \quad \lambda = 100.$$

Pourquoi les deux ? La L_1 seule prédit la « moyenne » des sorties possibles $\Rightarrow$ **flou** ; l'adversarial seul est net mais infidèle. Avec $\lambda = 100$, un écart pixel moyen de 0.01 pèse déjà 1.0 : la **fidélité domine**, l'adversarial **affûte**.

PatchGAN : un discriminateur local

D ne rend pas **un** verdict global mais une **carte** de verdicts, chacun ne voyant qu'un **patch** de l'image (ex. 70×70) ; la loss moyenne sur les patches.

- hypothèse : la **texture est locale** — juger des patches suffit ;
- moins de paramètres, applicable à toute taille d'image ;
- la cohérence globale est déjà garantie par la L_1 .

D'où viennent les paires ?

- annotées : IRM $\rightarrow$ CT, plans $\rightarrow$ façades ;
- **gratuites**, par dégradation calculable : colorisation (gris $Y = 0.299R + 0.587G + 0.114B \rightarrow$ couleur), inpainting (masque), restauration.

Sans paires alignées ?

CycleGAN (Zhu et al., 2017) : deux GAN couplés $G : X \rightarrow Y$, $F : Y \rightarrow X$ + contrainte de **cycle-consistance** $F(G(x)) \approx x$ — traduction de domaine non supervisée.

SRGAN — super-résolution perceptuelle

Le problème du MSE seul (Ledig et al., 2017)

Optimiser l'écart pixel produit des images **lisses et floues** : les textures plausibles sont « moyennées ». SRGAN compare les images **ailleurs que dans les pixels** :

$$\ell^{SR} = \underbrace{\left\| \phi(I^{HR}) - \phi(G(I^{LR})) \right\|^2}_{\substack{\text{content loss: écart entre } \textit{features} \\ \phi = \text{CNN pré-entraîné gelé (VGG19)}}} + 10^{-3} \underbrace{\mathcal{L}_{\text{adv}}(G)}_{\substack{\text{loss adversariale:} \\ \text{réalisme des textures}}}$$

Terme à terme : ϕ transforme chaque image en carte de features (textures, structures) — on compare des **percepts**, pas des pixels ; le facteur 10^{-3} fait de l'adversarial un **assaisonnement** : le contenu domine, le réalisme ajuste.

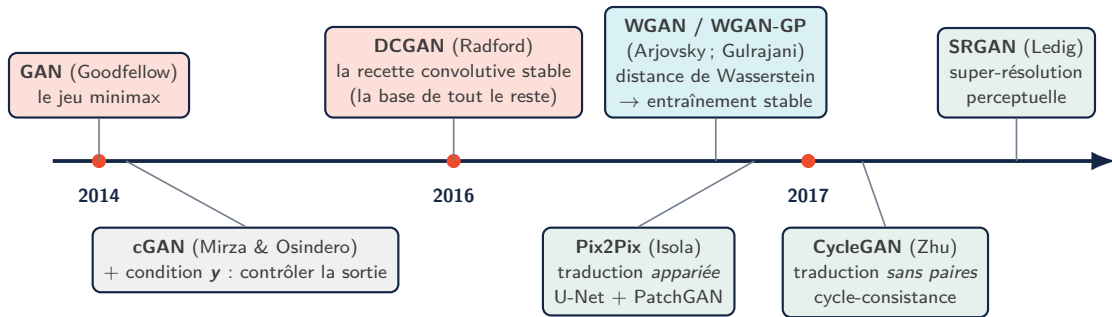
Recette en 2 temps + architecture

- 1) pré-entraîner G seul en loss pixel (**SRResNet**) : fidélité maximale ;
- 2) repartir de ce G et ajouter perceptuel + adversarial : les textures ;
- G : blocs résiduels + upsampling **sub-pixel nn.PixelShuffle** ;
 D : CNN classique ou PatchGAN.

Applications & compromis

- super-résolution d'images **médicales** (CT basse dose, IRM rapide) ;
- imagerie **satellite** (Sentinel-2) et vidéosurveillance ;
- compromis **perception-distorsion** : le PSNR baisse un peu, le réalisme perçu monte — d'où les métriques de la slide suivante.

Les variantes de GAN — qui apporte quoi ?



La grille de lecture à retenir

Une variante = une réponse à un problème précis : jeu instable → **WGAN-GP** ; contrôler *quoi* générer → **cGAN** ; image → image avec paires → **Pix2Pix**, sans paires → **CycleGAN** ; augmenter la résolution → **SRGAN**. Après 2018 (StyleGAN...), la diffusion prend le relais pour la synthèse pure — mais ces briques adversariales restent utilisées (super-résolution temps réel, distillation de modèles de diffusion).

Métriques des modèles génératifs : PSNR, SSIM, IS, FID

Fidélité à une référence (reconstruction, SR)

PSNR — rapport signal/bruit (en dB), **plus haut = mieux** :

$$\text{PSNR} = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}} \xrightarrow[\text{MAX}=1]{\text{MSE}=10^{-3}} 30 \text{ dB}$$

SSIM — compare ce que l'œil perçoit, comme **produit de 3 facteurs** $\in [0, 1]$:

$$\text{SSIM} = \underbrace{l(x, y)}_{\text{luminance}} \cdot \underbrace{c(x, y)}_{\text{contraste}} \cdot \underbrace{s(x, y)}_{\text{structure}}$$

SSIM = 1 si images identiques — bien plus proche du jugement humain que le PSNR.

Qualité d'une distribution générée (GAN, diffusion)

FID — distance entre les **features Inception-v3** des images réelles (r) et générées (g), **plus bas = mieux** :

$$\text{FID} = \underbrace{\|\mu_r - \mu_g\|^2}_{\text{écart des moyennes}} + \underbrace{\text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})}_{\text{écart des covariances}}$$

Sensible à la qualité et à la diversité : le mode collapse fait **exploser** le FID.

IS (*Inception Score*) : mesure si les images sont nettes et variées — historique, supplanté par le FID (qui compare au réel).

Outils PyTorch

Métrique	Outil	Quand l'utiliser
PSNR / SSIM	<code>torchmetrics</code> (<code>PeakSignalNoiseRatio</code> , <code>SSIM</code>)	débruitage, super-résolution, traduction alignée
FID	<code>pytorch-fid</code> , <code>torchmetrics.image.fid</code>	GAN / diffusion : qualité + diversité ($\geq 10k$ images)
IS	<code>torchmetrics.image.inception</code>	historique ; préférer le FID (compare au réel)

GAN — cas d'usage en Computer Vision

Quelle variante pour quel problème ?

Cas d'usage	Variante	Métriques	Exemple concret
Synthèse pure	DCGAN, WGAN-GP	FID, IS	visages, textures, augmentation
Augmentation médicale	cGAN (par classe)	FID + macro-F1 aval	lésions dermato rares (IRM, dermato)
Traduction alignée	Pix2Pix (U-Net + Patch-GAN)	SSIM, FID	IRM → CT, plans → photos
Traduction non alignée	CycleGAN	FID	domaine A ↔ B sans paires
Super-résolution	SRGAN / ESRGAN	PSNR, SSIM, FID	CT basse dose, satellite, vidéo
Restauration / inpainting	Pix2Pix, GAN + masque	PSNR, SSIM	images dégradées, artefacts

VAE, GAN, diffusion — le panorama génératif complet

Trois façons d'apprendre à générer (synthèse des Modules 3 & 4)

	VAE	GAN	Diffusion
Principe	maximiser l'ELBO	jeu minimax G vs D	régression de débruitage
Densité $p_\theta(x)$	approchée (borne)	implicite (aucune)	approchée (borne)
Échantillonnage	1 passe décodeur	1 passe G	T passes (DDPM) / S (DDIM)
Rendu	flou (moyennes)	net (hautes fréquences)	état de l'art
Diversité (modes)	bonne	mode collapse possible	excellente
Stabilité d'entraînement	stable (une loss)	équilibre de Nash délicat	stable (MSE)
Espace latent	structuré, interpolable	interpolable	trajectoire de bruit

En pratique

La **diffusion** domine la synthèse pure (qualité, diversité) mais paie ses S passes réseau ; le **GAN** reste le choix quand la **latence** compte (super-résolution temps réel, traduction embarquée, augmentation massive) et comme **brique** : la loss adversariale se greffe sur tout modèle de restauration, et la distillation adversariale compresse aujourd'hui la diffusion vers 1–4 pas.

Vision Transformers : ViT, Swin & SegFormer

De la convolution à l'attention

Ce que « sait » un CNN (biais inductifs)

- **localité** : un pixel interagit avec ses voisins ;
- **invariance par translation** : mêmes poids partout ;
- champ réceptif **croissant avec la profondeur** — l'information lointaine met des couches à se rejoindre.

Ces a priori sont une **force** quand les données sont rares... et une **limite** quand elles abondent.

Le pari du Transformer (Dosovitskiy et al., 2021)

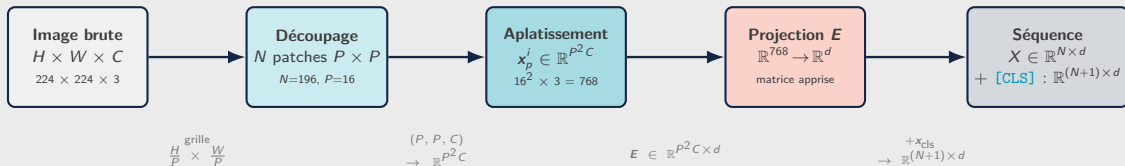
- **aucun** biais spatial : l'image est une **séquence de patches** ;
- chaque patch peut attendre sur **tous** les autres dès la couche 1 (champ réceptif global immédiat) ;
- les relations utiles sont **appprises**, pas imposées — à condition d'avoir assez de données (ou un pré-entraînement massif).

Le message empirique

À petit dataset : CNN > ViT (les biais inductifs compensent). Avec un grand pré-entraînement (ImageNet-21k, JFT) : ViT $\geq$ CNN, et **le fine-tuning d'un ViT pré-entraîné** devient le choix par défaut — exactement le réflexe transfer learning du Module 2.

ViT — du pixel au token : le patch embedding

L'entrée du ViT n'est pas une image brute — c'est une séquence de vecteurs d'embedding



Calcul à la main — ViT-Base/16, image $224 \times 224 \times 3$

$$N = \frac{H}{P} \times \frac{W}{P} = \frac{224}{16} \times \frac{224}{16} = 196 \text{ patches}$$

$$\text{dim. patch aplati} = P^2 \cdot C = 16^2 \times 3 = 768$$

$$X \in \mathbb{R}^{196 \times d} \xrightarrow{+ [\text{CLS}]} X \in \mathbb{R}^{197 \times d}$$

En une seule opération PyTorch

Découpage + projection = `nn.Conv2d(3, d, kernel_size=P, stride=P)` : une convolution de noyau et de pas P visite chaque patch **une fois, sans chevauchement** — c'est l'implémentation canonique.

Analogie NLP : le patch comme un mot

En NLP, un Transformer reçoit une séquence de vecteurs de mots. Le ViT fait la même chose avec des **vecteurs de patches**. La matrice E joue le rôle de la table d'embedding : elle apprend quelle information de chaque région est pertinente.

Point clé

$X \in \mathbb{R}^{N \times d}$ n'est **pas** une matrice de pixels bruts : c'est une **séquence de vecteurs d'embedding de patches**, permettant d'appliquer l'attention exactement comme en NLP.

Self-attention — Q, K, V : le mécanisme

Intuition : trois rôles par token

Chaque token joue **trois rôles** simultanément :

- **Q** (*Query*)
"que cherche-je ?"
— vecteur de recherche ;
- **K** (*Key*)
"de quoi parle-je ?"
— étiquette du token ;
- **V** (*Value*)
"quelle info je fournis ?"
— contenu à transmettre.

Pourquoi $\sqrt{d_k}$?

Si $\mathbf{q}, \mathbf{k} \sim \mathcal{N}(0, 1)$ composante à composante, $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$: les scores croissent avec d_k , le softmax **sature** (gradients ≈ 0). Diviser par $\sqrt{d_k}$ ramène la variance à 1.

Scaled dot-product attention (Vaswani et al., 2017)

Pour $\mathbf{X} \in \mathbb{R}^{n \times d}$, trois projections apprises ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d_k}$) :

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)}_{\text{poids } A \in \mathbb{R}^{n \times n}} \mathbf{V}$$

Lecture étape par étape :

1. $\mathbf{Q}\mathbf{K}^\top$: matrice $n \times n$ des scores — $\text{score}(i, j) = \mathbf{q}_i \cdot \mathbf{k}_j$ mesure l'affinité entre tokens i et j ;
2. $\div \sqrt{d_k}$: évite la saturation du softmax quand d_k est grand (voir ci-dessous) ;
3. softmax (par ligne) : transforme les scores en poids ≥ 0 qui somment à 1 ;
4. $\times \mathbf{V}$: chaque token reçoit une **moyenne pondérée** du contenu de tous les tokens.

Coût quadratique en n

$\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$: coût $\mathcal{O}(n^2 d_k)$. Pour $n=196$ (ViT-Base/16) — raisonnable. En haute résolution ($n \gg 1000$) $\Rightarrow$ **Swin** (fenêtres) ou **SegFormer** (réduction).

Self-attention — exemple numérique complet

Mini-séquence : $n = 2$ tokens, $d_k = 2$

$$q_1 = (1, 0), \quad k_1 = (1, 0), \quad k_2 = (0, 1), \quad v_1 = (1, 0), \quad v_2 = (0, 2).$$

1) Scores (produit scalaire $\div \sqrt{d_k} = \sqrt{2}$) :

$$s_{11} = \frac{q_1 \cdot k_1}{\sqrt{2}} = 0.707, \quad s_{12} = \frac{q_1 \cdot k_2}{\sqrt{2}} = 0.$$

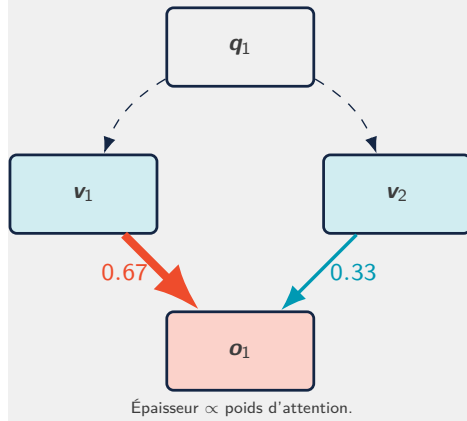
2) Softmax (scores $\rightarrow$ poids qui somment à 1) :

$$a_{11} = \frac{e^{0.707}}{e^{0.707} + e^0} = \mathbf{0.670}, \quad a_{12} = \mathbf{0.330}.$$

3) Sortie (moyenne pondérée des valeurs) :

$$o_1 = 0.670 (1, 0) + 0.330 (0, 2) = \mathbf{(0.670, 0.660)}.$$

Routage de l'information



Idée clé

Poids dépendant du **contenu** (vs convolution à poids fixes) :
un **routage adaptatif**.

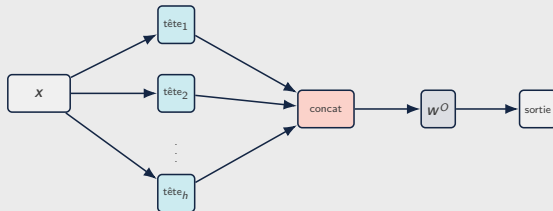
Multi-Head Attention

h têtes en parallèle

Une seule attention = un seul « type de relation ». On projette vers h sous-espaces de dimension $d_k = d/h$, chaque tête fait son attention, puis on **concatène** et on reprojette :

$$\text{head}_i = \text{Attention}(xW_i^Q, xW_i^K, xW_i^V)$$

$$\text{MHA}(\mathbf{X}) = [\text{head}_1; \dots; \text{head}_h] W^O$$



Même coût total qu'une grosse tête, mais h **relations apprises indépendamment** (texture, contours, position...).

```
import torch.nn as nn
```

```
mha = nn.MultiheadAttention(  
    embed_dim=768, num_heads=12,  
    batch_first=True)
```

```
x = torch.randn(8, 197, 768) # (B, n, d)  
out, attn = mha(x, x, x, # self-attention  
                need_weights=True)  
print(out.shape) # (8, 197, 768)  
print(attn.shape) # (8, 197, 197)
```

À noter

`attn` est la carte d'attention moyenne — celle que l'on visualise pour l'**interprétabilité** (attention maps, cf. XAI du Module 2).

ViT-Base

$d = 768$, $h = 12 \Rightarrow d_k = 64$ par tête.

ViT — les équations du Transformer, terme à terme

Étape 0 — construire la séquence d'entrée $z_0 \in \mathbb{R}^{(N+1) \times d}$

$$z_0 = \left[\underbrace{x_{\text{cls}}}_{\substack{\text{token [CLS] appris} \\ \text{(agrège le global, comme BERT)}}} ; \underbrace{x_p^1 E}_{\text{patch 1 projeté}} ; \dots ; \underbrace{x_p^N E}_{\text{patch } N \text{ projeté}} \right] + \underbrace{E_{\text{pos}}}_{\substack{\text{position apprise — sans elle,} \\ \text{les patches sont un sac non ordonné}}}$$

Étapes 1 à L — le même bloc, répété

Pour $\ell = 1, \dots, L$:

$$z'_\ell = \underbrace{\text{MHA}}_{\text{self-attention}} \left(\underbrace{\text{LN}(z_{\ell-1})}_{\text{norm. pré-bloc}} \right) + \underbrace{z_{\ell-1}}_{\text{résidu}}$$
$$z_\ell = \underbrace{\text{MLP}}_{\substack{2 \text{ couches, GELU,} \\ \text{expansion} \times 4}} \left(\text{LN}(z'_\ell) \right) + z'_\ell$$

Deux sous-étapes seulement : **mélanger les tokens entre eux** (MHA), puis **transformer chaque token individuellement** (MLP). Les **résidus** (« + z ») laissent le gradient traverser les L blocs, comme en ResNet.

Étape finale — prédiction

$$\hat{y} = \text{LN} \left(\underbrace{z_L^0}_{\substack{\text{token [CLS]} \\ \text{à la couche } L}} \right) W_{\text{head}}$$

Seul le token **[CLS]** (indice 0) sert à la classification. En PyTorch : `nn.TransformerEncoderLayer(norm_first=True)`.

Calcul à la main — d'où viennent les 86M de ViT-Base ?

$d = 768$, $L = 12$ blocs. Par bloc : attention (W^Q, W^K, W^V, W^O) $\approx 4d^2$; MLP ($d \rightarrow 4d \rightarrow d$) $\approx 8d^2$; soit $12d^2 \approx 7.1\text{M}$.

$$12 \text{ blocs} \times 12d^2 = 144 \times 768^2 \approx \mathbf{85\text{M}}$$

+ embeddings (E, E_{pos}) $\approx \mathbf{86\text{M}}$. ✓

ViT — où regarde le modèle ? L'attention rollout

L'interprétabilité native du Transformer

La ligne [CLS] de la carte d'attention $\mathbf{A} \in \mathbb{R}^{n \times n}$ dit **combien chaque patch contribue à la décision** — sans gradient ni rétropropagation, contrairement à Grad-CAM (Module 2). **Mais** regarder **une seule couche** ment : à chaque bloc, le résidu « +z » fait passer la moitié de l'information **sans attention**.

Attention rollout (Abnar & Zuidema, 2020)

On corrige chaque couche, puis on suit le flux à travers les L blocs :

$$\tilde{\mathbf{A}}_\ell = \underbrace{\frac{1}{2} \mathbf{A}_\ell}_{\text{moitié du flux : l'attention}} + \underbrace{\frac{1}{2} \mathbf{I}}_{\text{moitié du flux : le résidu}}, \quad \mathbf{R} = \underbrace{\tilde{\mathbf{A}}_L \cdots \tilde{\mathbf{A}}_1}_{\text{produit = suivre l'information de l'entrée à la sortie}}$$

$\mathbf{A}_\ell$: carte d'attention du bloc ℓ , moyennée sur les têtes. La ligne [CLS] de $\mathbf{R}$, repliée en grille de patches et suréchantillonnée, donne la **heatmap** « où le modèle regarde ».

Calcul à la main — 1 bloc, 2 tokens

Attention brute du [CLS] (token 1) : 90 % sur lui-même, 10 % sur le patch :

$$\mathbf{A} = \begin{pmatrix} 0.9 & 0.1 \\ 0.4 & 0.6 \end{pmatrix} \Rightarrow \tilde{\mathbf{A}} = \frac{1}{2} \mathbf{A} + \frac{1}{2} \mathbf{I} = \begin{pmatrix} 0.95 & 0.05 \\ 0.2 & 0.8 \end{pmatrix}$$

L'attention brute disait « 10 % du patch » ; le rollout corrige à 5 % : la moitié du flux est passée par le résidu, **sans regarder personne**.

À retenir

- complète Grad-CAM / SHAP (Module 2) pour l'audit des modèles ;
- spectaculaire avec **DINO** (ViT auto-supervisé) : les cartes segmentent les objets **sans aucun label**.

Swin Transformer — attention par fenêtres décalées

Le problème du ViT en haute résolution

L'attention globale coûte $\mathcal{O}(n^2)$ en nombre de patches : impraticable en haute résolution. Swin (Liu et al., 2021) la restreint à des **fenêtres locales** de taille fixe $M \times M$ ($M=7$).

Gain de complexité (pour $h \times w$ patches de dimension C , terme dominant) :

$$\underbrace{2(hw)^2 C}_{\text{attention globale, quadratique en } hw} \xrightarrow{\text{Swin}} \underbrace{2M^2 hw C}_{\text{attention fenêtrée, linéaire en } hw}$$

Le facteur $(hw)^2$ est remplacé par $M^2 \cdot hw$: puisque $M=7$ est fixe, le coût devient **linéaire** en hw .

Exemple chiffré (stage 1 : 56×56 patches, $C=96$)

Terme d'attention : $2(hw)^2 C = 2 \cdot 3136^2 \cdot 96 \approx 1.9 \cdot 10^{12}$

Fenêtré : $2M^2 hw C = 2 \cdot 49 \cdot 3136 \cdot 96 \approx 3.0 \cdot 10^7$

$\Rightarrow$ rapport = $(hw)/M^2 = 3136/49 : 64 \times$ **moins de calcul**.

Fenêtres décalées (SW-MSA)

Des fenêtres fixes ne communiquent jamais entre elles. Un bloc sur deux **décale** la grille de $\lfloor M/2 \rfloor$: les frontières changent, l'information **circule entre fenêtres** (implémenté par décalage cyclique `torch.roll` + masques).

Hiérarchie à la CNN

- 4 étages : résolutions $\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}$ (**patch merging** $2 \times 2 \rightarrow$ canaux $\times 2$) ;
- produit des **cartes multi-échelles** — réutilisables par FPN, U-Net, têtes de détection/segmentation ;
- biais de position **relatif** appris B ajouté aux scores : $\text{softmax}(QK^T / \sqrt{d} + B)V$;
- le ViT « backbone généraliste » : classification, détection, segmentation.

SegFormer — segmentation sémantique efficace

Encodeur MiT (Mix Transformer)

- hiérarchique (4 étages, comme Swin) ;
- **attention à séquence réduite** : K, V sous-échantillonnés d'un facteur $R \Rightarrow$ coût $\mathcal{O}(n^2/R)$;
- **pas de positional encoding** : une **Conv** 3×3 dans le MLP (Mix-FFN) capture la position $\Rightarrow$ robuste aux changements de résolution ;
- décodeur **tout-MLP** : fusion des 4 échelles, léger et rapide.

Métriques de segmentation — terme à terme

$$\text{IoU} = \frac{\overbrace{\text{TP}}^{\text{intersection}}}{\underbrace{\text{TP} + \text{FP} + \text{FN}}_{\text{union}}} \quad \text{Dice} = \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}}$$

mIoU = moyenne de l'IoU sur les K classes (`torchmetrics.JaccardIndex`) ; le Dice, plus indulgent, domine en **imagerie médicale**.

Calcul à la main

Masque prédit vs vérité : TP=6, FP=2, FN=4 pixels : $\text{IoU} = \frac{6}{6+2+4} = \mathbf{0.50}$;
 $\text{Dice} = \frac{12}{12+2+4} = \mathbf{0.67}$.

```
from transformers import (
    SegformerForSemanticSegmentation,
    SegformerImageProcessor)

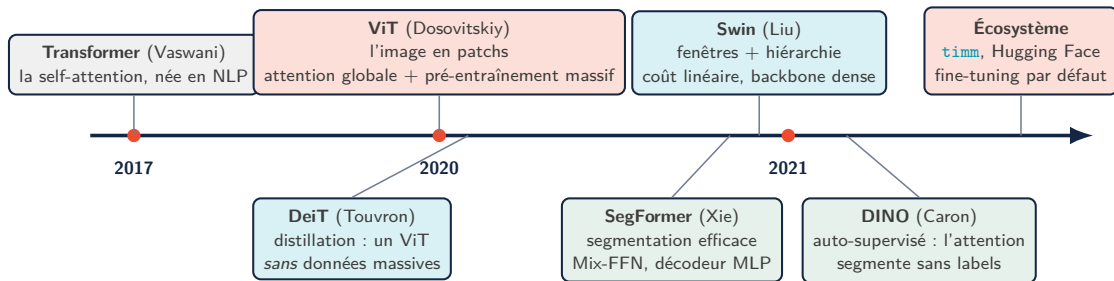
name = ("nvidia/segformer-b2-"
        "finetuned-ade-512-512")
proc = SegformerImageProcessor.from_pretrained(name)
model = SegformerForSemanticSegmentation \
        .from_pretrained(name)

inputs = proc(images=img, return_tensors="pt")
logits = model(**inputs).logits # (B,K,H/4,W/4)
mask = logits.argmax(dim=1)     # classes par pixel
```

Applications

Segmentation d'organes/tumeurs (IRM, CT), rétine ; scènes urbaines (CityScapes) ; bâtiments et routes (satellite).

Les variantes de ViT — qui apporte quoi ?



La grille de lecture à retenir

Une variante = une réponse à un problème précis : peu de données → **DeiT** (ou fine-tuning d'un pré-entraîné) ; haute résolution / tâches denses → **Swin** ; segmentation → **SegFormer** ; pas de labels → **DINO**. Le ViT « pur » reste la référence quand le pré-entraînement massif est disponible — et la brique de base de CLIP, SAM et des modèles multimodaux actuels.

CNN vs ViT — quel modèle pour quelles données ?

Comparaison synthétique

Critère	CNN (ResNet, EfficientNet)	ViT / Swin
Biais inductifs	localité, translation (forts)	quasi aucun (appris)
Données nécessaires	modestes (qq. milliers)	grandes, ou pré-entraînement massif
Champ réceptif	local, croît avec la profondeur	global dès la 1^{re} couche
Coût en résolution	linéaire en pixels	quadratique (ViT) → linéaire (Swin)
Interprétabilité	Grad-CAM (Module 2)	attention maps natives
Robustesse aux occlusions / textures	sensible à la texture	plus orienté forme , souvent plus robuste

Règle pratique — et le verdict empirique

Dataset < 10k images, pas de pré-entraînement adapté ⇒ **CNN fine-tuné** (Module 2). Pré-entraînement disponible (ImageNet-21k, DINO) ou tâche dense en haute résolution ⇒ **ViT/Swin fine-tuné via timm**, SegFormer pour la segmentation.

Vérifié à budget égal : sur ≈ 7 000 images médicales **sans pré-entraînement**, un petit CNN **tient tête, voire dépasse** un ViT ou un Swin entraînés de zéro — ses biais inductifs sont un a priori **gratuit**. Un backbone **pré-entraîné** (**timm**) renverse le verdict.

Écosystème pratique : timm & Hugging Face

```
import timm

# ViT pre-entraîne, tête remplacée (7 classes)
model = timm.create_model(
    "vit_base_patch16_224",
    pretrained=True, num_classes=7)

# Swin pour la classification fine-grained
swin = timm.create_model(
    "swin_base_patch4_window7_224",
    pretrained=True, num_classes=7)

# Fine-tuning : memes recettes qu'au Module 2
# (gel des couches, lr différentiel, OneCycleLR)
for p in model.blocks[:8].parameters():
    p.requires_grad = False
```

Ce que fournit l'écosystème

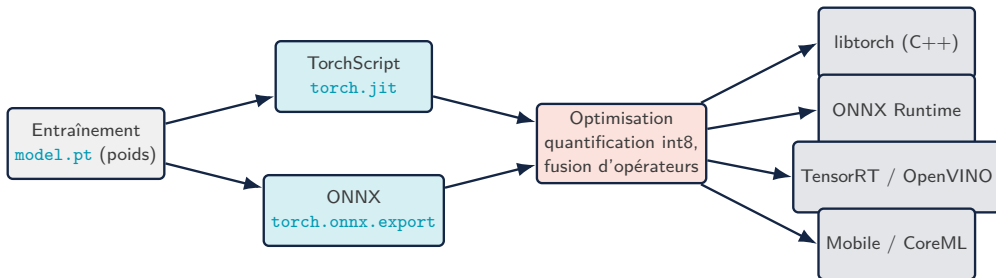
- `timm` : des centaines de backbones (ViT, Swin, DeiT, EfficientNet) pré-entraînés, API uniforme ;
- `transformers` (HF) : ViT, SegFormer, DINO + processors d'entrée ;
- `einops` : `rearrange(x, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)')` — le patch embedding en une ligne lisible.

Interprétabilité incluse

L'*attention rollout* (vue plus haut) s'applique telle quelle à ces backbones : un audit natif du modèle, en complément de Grad-CAM / SHAP (Module 2).

Déploiement : TorchScript, ONNX & quantification

Du notebook à la production — vue d'ensemble



Pourquoi exporter ? Le checkpoint .pt ne suffit pas

Un `state_dict` exige **Python + la classe du modèle** pour être rechargé. La production veut un artefact **autonome** : exécutable sans Python (serveur C++, GPU embarqué, mobile), au **graphe figé** (optimisable, versionnable, auditable) et **rapide** (fusion d'opérateurs, précision réduite).

Deux familles d'export

TorchScript : reste dans l'écosystème PyTorch (libtorch). **ONNX** : format **ouvert** interopérable (TensorRT, OpenVINO, CoreML, navigateur).

Règle d'or

Toujours **valider numériquement** l'export : mêmes entrées $\Rightarrow$ sorties identiques (à 10^{-5} près) entre le modèle Python et l'artefact exporté.

TorchScript — `trace` vs `script`

```
model.eval()
x = torch.randn(1, 3, 224, 224)

# 1) TRACE : enregistre les ops exécutées
traced = torch.jit.trace(model, x)

# 2) SCRIPT : compile un sous-ensemble Python
scripted = torch.jit.script(model)

traced.save("model_traced.pt")
m = torch.jit.load("model_traced.pt")

# validation numérique de l'export
assert torch.allclose(model(x), m(x),
                       atol=1e-5)
```

Quelle méthode choisir ?

	<code>trace</code>	<code>script</code>
Principe	exécute et enregistre	compile le code
Contrôle de flux (<code>if/for</code>)	figé sur l'exemple	préservé
Limites	branches dépendant des données	sous-ensemble de Python

Piège classique

Un `if x.mean() > 0:` dans le `forward` est **silencieusement figé** par `trace` (seule la branche prise à l'export survit) — utiliser `script`, ou vérifier avec des entrées couvrant les deux branches.

Et ensuite ?

L'artefact `.pt` se charge en **C++** (`torch::jit::load`) — aucun interpréteur Python en production.

ONNX — le format d'échange interopérable

```
torch.onnx.export(  
    model, x, "model.onnx",  
    opset_version=17,  
    input_names=["image"],  
    output_names=["logits"],  
    dynamic_axes={"image": {0: "batch"},  
                  "logits": {0: "batch"}})  
  
import onnxruntime as ort  
sess = ort.InferenceSession(  
    "model.onnx",  
    providers=["CPUExecutionProvider"])  
out = sess.run(None,  
    {"image": x.numpy()})[0]  
  
assert np.allclose(out,  
    model(x).detach().numpy(), atol=1e-5)
```

Points clés de l'export

- `opset_version` : version du jeu d'opérateurs ONNX (fixe la compatibilité runtime) ;
- `dynamic_axes` : batch variable à l'inférence (sinon la taille de l'exemple est **figée**) ;
- l'export **trace** le modèle : mêmes précautions de contrôle de flux que `jit.trace`.

Runtimes cibles

ONNX Runtime (CPU/GPU, serveur & edge), **TensorRT** (GPU NVIDIA, fusion + FP16/INT8), **OpenVINO** (CPU Intel), **CoreML** (Apple). Un seul artefact, plusieurs cibles matérielles.

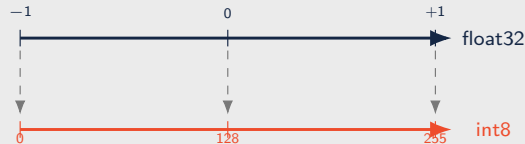
Quantification & PyTorch Mobile

Principe : remplacer chaque float32 par un entier int8 (sur 256 niveaux)

On encode un réel x par un entier $q \in [0, 255]$, puis on le décode :

$$q = \left\lfloor \frac{x}{s} \right\rfloor + z \quad \hat{x} = s(q - z)$$

- $s = \text{échelle}$ (taille d'un pas) = $\frac{x_{\max} - x_{\min}}{255}$;
- $z = \text{zéro-point}$ (l'entier qui code $x=0$) ;
- $\lfloor \cdot \rfloor = \text{arrondi}$; $\hat{x} = \text{valeur reconstruite}$ ($\approx x$).



$$s = \frac{2}{255} \approx 0.008, \quad z = 128$$

Erreur max de quantification = $s/2 \approx 0.004$ — négligeable pour la plupart des classifieurs.

Trois stratégies

- **dynamique** : poids int8, activations quantifiées à la volée — 1 ligne, idéal Linear/LSTM ;
- **statique** (PTQ) : calibration sur quelques batches → échelles d'activations figées ;
- **QAT** : fausse quantification pendant le fine-tuning — la meilleure précision finale.

```
qmodel = torch.quantization.quantize_dynamic(  
    model, {nn.Linear}, dtype=torch.qint8)
```

Gains typiques (CPU)

Critère	float32	int8
Taille modèle	$\times 1$	$\approx \times 1/4$
Latence CPU	$\times 1$	$\times 2$ à $\times 4$ plus rapide
Précision	référence	-0 à -1 pt (PTQ)

PyTorch Mobile

`torch.utils.mobile_optimizer.optimize_for_mobile(scripted)` : fusion Conv+BN+ReLU, nettoyage du graphe → artefact `.pt1` pour **iOS / Android** — à combiner avec la quantification.

Projet fil rouge & synthèse

Projet fil rouge — organisation & thèmes

Format

Équipes de **2–3 participants** ; un **pipeline DL complet** sur un cas réel, initié dès le Module 1 et enrichi à chaque module (données → modèle → évaluation → interprétabilité → export).

Thèmes proposés

Thème	Architectures	Métriques
CV médicale : classification / segmentation (IRM cérébrale, rétinopathie)	CNN, ViT, U-Net / SegFormer	accuracy, F1, mIoU, Dice
Contrôle qualité industriel : défauts + localisation	CNN + CAM / Grad-CAM	F1, mAP
Maintenance prédictive : RUL turbofan (NASA C-MAPSS)	LSTM + CNN 1D	MAE, RMSE
Super-résolution satellite (Sentinel-2)	SRGAN ou diffusion	PSNR, SSIM, FID
Segmentation urbaine (CityScapes)	SegFormer	mIoU
Génération médicale par diffusion (IRM/CT)	DDPM conditionnel	FID + F1 aval
Audit XAI complet d'un classifieur médical	Grad-CAM + SHAP + rapport	fidélité, plausibilité

Critères d'évaluation

1. **Pertinence du choix architectural** et justification (pourquoi ce modèle pour ces données ?) ;
2. **Qualité du pipeline** : prétraitement, augmentation, validation croisée, reproductibilité (seeds) ;
3. **Performance des métriques** adaptées au cas d'usage (accuracy, mAP, IoU, SSIM...) ;
4. **Clarté de la présentation** et démo live.

Livrables attendus

- dépôt de code structuré (données **non commitées**, README, checkpoints nommés) ;
- notebook ou scripts reproductibles de bout en bout ;
- un volet **interprétabilité** (Grad-CAM / SHAP / attention maps) ;
- un **export d'inférence** (TorchScript ou ONNX) validé numériquement ;
- présentation finale avec démo.

Conseil

Réutiliser les briques des TP : pipelines DermaMNIST, boucles d'entraînement, conventions de checkpoints — le projet **assemble**, il ne repart pas de zéro.

Synthèse mathématique du Module 4

Concept	Formule clé	PyTorch / outil
GAN minimax	$\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$	2 <code>backward()</code> , <code>detach()</code>
D optimal	$D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$; G minimise la JSD	<code>BCEWithLogitsLoss</code>
Équilibre	$D(\mathbf{x}) \approx D(G(\mathbf{z})) \approx 0.5$, $\mathcal{L}_D \approx 2 \log 2$	suivi des courbes + grilles $\mathbf{z}_{\text{fix}}$
Loss non saturante	G maximise $\log D(G(\mathbf{z}))$	labels <code>ones</code> sur les fausses
WGAN-GP	$\sup_{\ f\ _L \leq 1} \mathbb{E}f(\mathbf{x}) - \mathbb{E}f(\tilde{\mathbf{x}}) + \lambda \mathbb{E}(\ \nabla f\ - 1)^2$	critique sans sigmoïde
cGAN	$D(\mathbf{x} \mathbf{y}), G(\mathbf{z} \mathbf{y})$	<code>nn.Embedding</code> concaténé
Pix2Pix	$\mathcal{L}_{\text{cGAN}} + \lambda \mathbb{E}\ \mathbf{y} - G(\mathbf{x})\ _1$, $\lambda=100$	U-Net + PatchGAN
SRGAN	loss VGG (features) + 10^{-3} adversariale	<code>nn.PixelShuffle</code> , VGG gelé
FID	$\ \boldsymbol{\mu}_r - \boldsymbol{\mu}_g\ ^2 + \text{Tr}(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g)^{1/2})$	<code>pytorch-fid</code>
Self-attention	$\text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}) \mathbf{V}$	<code>nn.MultiheadAttention</code>
ViT	$\mathbf{z}_0 = [\mathbf{x}_{\text{cls}}; \mathbf{x}_p \mathbf{E}] + \mathbf{E}_{\text{pos}}$, blocs MHA/MLP pré-norm	<code>timm.create_model</code>
Attention rollout	$\tilde{\mathbf{A}}_\ell = \frac{1}{2} \mathbf{A}_\ell + \frac{1}{2} \mathbf{I}$, $\mathbf{R} = \tilde{\mathbf{A}}_L \cdots \tilde{\mathbf{A}}_1$	ligne <code>[CLS]</code> → heatmap
Swin (W-MSA)	$2M^2 hwC$ au lieu de $2(hw)^2 C$	<code>swin_base_patch4...</code>
SegFormer / mIoU	$\text{mIoU} = \frac{1}{K} \sum_k \text{TP}_k / (\text{TP}_k + \text{FP}_k + \text{FN}_k)$	<code>transformers</code> (HF)
Quantification	$q = \lfloor x/s \rfloor + z$, $\hat{x} = s(q - z)$	<code>quantize_dynamic</code>

Trois ouvrages couvrent les concepts présentés

1. **Atienza, R.** (2018). *Advanced Deep Learning with Keras*. Packt Publishing. ISBN 978-1-78862-941-6.
Chapitres pertinents : GAN (DCGAN, cGAN), WGAN, GAN désenchevêtrés, super-résolution.
2. **Vasilev, I.** (2019). *Python Deep Learning* (2^e éd.). Packt Publishing.
Chapitres pertinents : modèles génératifs (GAN), mécanismes d'attention.
3. **Buduma, N., Buduma, N., & Papa, J.** (2022). *Fundamentals of Deep Learning* (2^e éd.). O'Reilly Media. ISBN 978-1-492-08128-7.
Chapitres pertinents : modèles génératifs, attention & transformers, mise en pratique.

Les références couvrent les fondements théoriques de toutes les architectures vues dans la formation (Modules 1 à 4).

 bassem.benhamed@enetcom.usf.tn

Module 4 — GAN, Vision Transformers & Déploiement | **Bassem Ben Hamed**